Database quan hệ: MySQL, PostgreSQL và phục hồi

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Một database quan hệ thực sự làm gì?

MySQL/MariaDB và PostgreSQL lưu dữ liệu theo bảng, có schema, index, transaction và giao thức client/server. Một query đi từ ứng dụng qua connection tới engine, engine chọn cách tìm dữ liệu, đọc trang dữ liệu/bộ nhớ/đĩa, có thể lấy lock, rồi trả hàng. Khi request API chậm, database có thể là nút thắt do query thiếu index, quá nhiều kết nối, lock contention hoặc đĩa chậm. Tăng CPU app không giải quyết mọi nguyên nhân.

Primary key xác định một hàng. Index giúp tìm nhanh theo một số điều kiện nhưng tốn dung lượng và chi phí ghi; thêm index cho mọi cột là sai. Transaction nhóm thao tác cần nhất quán. Isolation level ảnh hưởng chuyện transaction này nhìn thấy thay đổi của transaction khác; không cần thuộc mọi anomaly ngay, nhưng phải biết race condition có thể xảy ra nếu hai request cùng cập nhật số dư hoặc số lượng tồn.

Lab SQL căn bản

Dùng SQLite của Notes API để học câu lệnh không tốn cài đặt. Tạo 10 ghi chú qua API. Mở database bằng công cụ SQLite phù hợp hoặc một script Python đọc-only; chạy **SELECT id, body FROM notes ORDER BY id**, rồi truy vấn theo **id**. Trước khi chạy **DELETE**, sao lưu file lab và xác nhận đúng đường dẫn. Viết một câu hỏi: "Có bao nhiêu ghi chú?" rồi dùng **SELECT COUNT(*) FROM notes**. Sau đó giải thích vì sao một app có thể trả 500 khi schema bảng chưa được tạo hoặc quyền ghi file bị chặn.

Khi chuyển sang MySQL/PostgreSQL, giữ cùng schema và câu hỏi nhưng học thêm user, quyền, connection, port, service, log, backup và restore. SQLite lab giúp hiểu SQL; nó không mô phỏng replication hay vận hành server nhiều người dùng.

Replication, backup và failover

Replication chuyển thay đổi từ nguồn sang bản sao. Nó có thể giúp đọc, bảo trì hoặc tăng khả dụng, nhưng bản sao có thể trễ và có thể nhận cả lỗi xóa dữ liệu. Backup là khả năng quay về dữ liệu ở thời điểm đã lưu; restore là bằng chứng backup đó dùng được. Nếu RPO là 15 phút, quy trình backup mỗi ngày không đáp ứng dù có replica. Nếu RTO là một giờ, cần đo cả thời gian phục hồi dữ liệu, ứng dụng và kết nối người dùng.

Trong MySQL, replication có source/replica và log thay đổi. Failover cần xác định bản replica nào đã bắt kịp, ngăn split-brain/ghi vào hai nguồn, chuyển client và kiểm tra dữ liệu. Trong PostgreSQL, streaming replication/WAL cũng có vấn đề lag và cách chọn leader. Chi tiết lệnh phụ thuộc phiên bản và cấu hình; học qua tài liệu chính thức của phiên bản đang dùng. Một hệ thống có replica không tự động có HA: còn health detection, promotion, routing và thử nghiệm.

Tình huống phỏng vấn

Primary hỏng, replica chậm 30 giây. Bạn cần hỏi dữ liệu mới nhất có giá trị thế nào, RPO đã cam kết, replica nào an toàn để promote, client sẽ trỏ về đâu và liệu có ghi từ primary cũ sau khi nó trở lại không. Câu "đổi DNS sang replica" bỏ qua dữ liệu, fencing và ứng dụng có kết nối cache. Hãy viết kế hoạch từng bước trên giấy trước khi làm lab failover.

Lab restore có thể kiểm chứng

Với database lab, tạo dữ liệu đánh dấu thời điểm A; backup; thêm dữ liệu B; restore vào database **khác**; truy vấn để xác nhận dữ liệu A có, B không. Ghi độ trễ, kích thước file và thời gian restore. Sau đó viết RPO quan sát được giữa A và B. Không restore đè database đang dùng; không coi một lệnh backup exit 0 là đủ.

Tuning hiệu năng MySQL/MariaDB/PostgreSQL

Tuning bắt đầu bằng đo. Lấy query chậm, tần suất, số hàng, plan thực thi và độ trễ theo thời gian. **EXPLAIN** hoặc công cụ tương đương cho thấy engine định đi qua index/scan thế nào. Một query chậm vì thiếu index khác với query bị lock bởi transaction dài. Connection pool cạn có thể làm app timeout dù DB CPU thấp. Đĩa đầy, I/O latency và memory pressure cũng là nguyên nhân. Thay một yếu tố rồi đo lại; không copy cấu hình "tối ưu" từ blog cho máy khác.

Bài tập phân tích

Bảng notes có một triệu hàng. Query **SELECT * FROM notes WHERE id = ?** thường nhanh nhờ primary key; query tìm **body LIKE '%abc%'** có thể cần scan lớn hoặc giải pháp search khác. Hãy nêu bằng chứng cần xem trước khi thêm index hay đổi hệ. Nếu app gọi query này 500 lần cho một trang, vấn đề có thể là cách app dùng DB (N+1) hơn là cấu hình database.

Backup và recovery nhiều lớp

Backup có thể là snapshot vật lý, logical dump hoặc continuous backup kèm log. Cần biết thời điểm nhất quán, retention, mã hóa/quyền, failure domain và khả năng restore sang phiên bản engine nào. Backup cùng đĩa với DB không chống mất đĩa; replica có thể nhân bản thao tác xóa sai. Nếu DB khôi phục về thời điểm cũ nhưng consumer offset ở thời điểm mới, dữ liệu có thể lệch; runbook phải chỉ cách đồng bộ hoặc replay có kiểm soát.

Diễn tập với Notes API: đặt RPO 15 phút, RTO 2 giờ; backup A, thêm ghi chú B, mô phỏng DB hỏng. Restore **vào file/instance mới**, xác nhận dữ liệu, đổi app lab sang DB mới, chạy smoke test. Ghi giờ bắt đầu-kết thúc, dữ liệu mất và điểm làm vượt RTO. Sau đó đề xuất cải tiến cụ thể về tần suất, tự động hóa hoặc vị trí bản sao.

Theo dõi một câu query từ API xuống đĩa

Request tới API lấy connection từ pool, gửi SQL, database parse/plan, đọc index hoặc scan bảng, có thể đợi lock/I/O, rồi trả rows. Vì vậy độ trễ DB có thể do query plan, thiếu index, lock, số connection, đĩa hoặc mạng. Chỉ xem CPU DB không đủ. **EXPLAIN** giúp thấy hướng thực thi; metric query latency và slow query log cho biết tần suất/tác động. Thêm index có thể giúp đọc nhưng làm ghi nặng và tốn dung lượng; phải đo trước-sau.

Lab SQL có đối chứng

Với Notes API lab, tạo 100-1000 ghi chú thử trong DB riêng. Chạy **SELECT COUNT(*)**, truy vấn **WHERE id=?**, xem schema và index. Tạo một query tìm theo body, đo thời gian lặp lại ở dữ liệu nhỏ và lớn; đừng kết luận hiệu năng production từ SQLite lab, mục tiêu là học cách đo và nêu giả thuyết. Khi chuyển MySQL/PostgreSQL, dùng **EXPLAIN** theo phiên bản, xem rows ước lượng/thực tế, index được chọn và lock/connection. Ghi một thay đổi duy nhất rồi đo lại. Tránh tạo index tùy tiện trên database dùng chung.

Replication, HA và backup là ba cơ chế khác nhau

Replica nhận thay đổi từ primary/source, có thể trễ. HA cần phát hiện lỗi, chọn leader, ngăn split-brain, chuyển client và xác nhận. Backup giữ điểm phục hồi; nó có thể bảo vệ trước thao tác xóa nhầm mà replica đã nhân bản. RPO là lượng dữ liệu tối đa chấp nhận mất; RTO là thời gian tối đa khôi phục. Nếu replica chậm 30 giây và RPO bằng 0, promote ngay có thể vi phạm cam kết. Nếu backup hàng ngày, RPO không thể 15 phút trừ khi có log/continuous backup và quy trình khôi phục tương ứng.

Diễn tập restore

Tạo dữ liệu A -> backup -> tạo dữ liệu B -> restore vào instance/file khác -> kiểm tra A có, B không. Đo thời gian từ lúc tuyên bố sự cố tới API có thể đọc/ghi trên DB khôi phục; đó mới gần với RTO, không chỉ thời gian chạy lệnh restore. Ghi quyền, credential, DNS/connection string và schema version. Nếu app mới không tương thích schema trong backup, phải có kế hoạch tương thích. Không restore đè dữ liệu đang hoạt động để thử nghiệm.

Case: DB CPU thấp, API timeout

API timeout 5 giây; DB CPU 20%; connection pool exhausted. Có thể app giữ connection quá lâu do transaction dài, query chờ lock hoặc pool size nhỏ so với concurrency. Thu số active/idle/waiting connections, transaction tuổi cao, lock wait, query latency và thay đổi gần nhất. Nếu chỉ tăng pool từ 20 lên 200, database có thể nhận thêm tải và chậm hơn. Giảm tác động có thể rollback query mới, giới hạn traffic hoặc sửa transaction; xác nhận bằng error rate và wait time. Đây là ví dụ phỏng vấn cần đi từ triệu chứng đến bằng chứng, không từ một metric tới kết luận.

Tách bốn câu hỏi: đúng dữ liệu, nhanh, sẵn sàng và khôi phục được

DB quan hệ giữ dữ liệu theo bảng, ràng buộc và transaction. **Đúng dữ liệu** cần khóa, isolation, constraint và logic app; **nhanh** cần query/index/capacity; **sẵn sàng** cần failover/replication; **khôi phục** cần backup và restore. Một replica khỏe không bảo đảm backup: thao tác **DELETE** nhầm có thể được nhân bản. Một backup tốt không bảo đảm HA: restore có thể mất hàng giờ. Một query nhanh ở staging nhỏ không bảo đảm nhanh ở production dữ liệu lớn. Đừng dùng từ "DB ổn" mà không nói tiêu chí nào.

Query plan và index qua một ví dụ

SELECT * FROM notes WHERE owner_id = ? ORDER BY created_at DESC LIMIT 20 trên bảng 10 triệu dòng có thể cần index phù hợp **(owner_id, created_at)** tùy engine, selectivity và cách sắp xếp. Index chỉ trên **created_at** có thể vẫn phải lọc nhiều owner; index nhiều cột thay đổi chi phí ghi/storage. Đọc **EXPLAIN**/actual execution stats theo engine để xem scan, rows, sort, join và latency; không thêm index chỉ vì cột xuất hiện trong WHERE. **SELECT *** lấy nhiều dữ liệu hơn cần; pagination offset lớn có thể đắt; connection pool quá lớn có thể làm DB quá tải dù CPU API thấp.

Transaction dài giữ lock và tăng contention. Khi API timeout, hỏi nó chờ connection pool, lock, disk I/O, network hay query CPU. MySQL/MariaDB/PostgreSQL có lệnh/metric khác; Oracle càng khác về kiến trúc/license/vận hành. Học mô hình chung trước, rồi đọc tài liệu đúng engine/version. Redis không thay DB quan hệ chỉ vì đọc nhanh; cache invalidation và consistency là bài toán riêng.

Replication và failover cần chứng minh bằng timeline

Giả sử primary nhận một giao dịch lúc 10:00:00, client thấy success lúc 10:00:01, replica lag 5 giây; primary mất lúc 10:00:02. Nếu failover sang replica chưa nhận giao dịch, có thể mất dữ liệu dù API đã trả success, tùy chế độ replication/ack. RPO thực phải đo theo cơ chế và tải. Read replica có thể trả dữ liệu cũ; nếu client vừa ghi rồi đọc từ replica, trải nghiệm không nhất quán. Failover còn cần chọn leader, đổi connection/DNS, tránh split-brain và kiểm tra app reconnect. Không gọi "có replication" là "HA xong".

MySQL point-in-time recovery thường cần full backup và binary log liên quan để replay tới thời điểm; chỉ giữ full backup hàng tuần khó đạt RPO phút. PostgreSQL dùng base backup và WAL archiving cho mục tiêu tương tự theo cấu hình. Lưu backup/log ở failure domain khác, mã hóa, quyền tối thiểu và retention phù hợp. Đọc MySQL 8.4 backup/recovery và PITR theo version thực.

Lab restore có kiểm chứng trên dữ liệu giả

Điều kiện: database lab riêng hoặc SQLite app mẫu, không dùng DB production. Tạo bản ghi A lúc T1, tạo backup nhất quán, tạo bản ghi B lúc T2. Restore vào **instance/đường khác**; kiểm tra schema, row count, checksum hoặc truy vấn nghiệp vụ: A có, B không có nếu backup ở giữa. Ghi thời gian từ bắt đầu restore đến smoke test API đọc/ghi thành công. Nếu dùng MySQL/PostgreSQL lab, thử thêm log/WAL để phục hồi tới gần T2 và xác định RPO thực. Dọn bản restore sau khi đã ghi kết quả; không ghi đè source DB để "thử nhanh".

Lỗi có chủ đích: app mới yêu cầu cột **priority**, migration chưa chạy. Pod Ready vì **/health** chỉ kiểm tra process; **POST /notes** trả 500. Bằng chứng: app log báo thiếu cột, schema version cũ, release diff.

Giảm tác động: rollback app nếu schema vẫn tương thích, hoặc chạy migration đã review/roll-forward theo runbook. Sau đó thêm integration test chạy migration và smoke test ghi dữ liệu. Chỉ tăng DB CPU không chữa schema thiếu.

Tuning theo giả thuyết

Đừng bắt đầu bằng "tăng max_connections". Nếu pool app mở 500 connection tới DB có giới hạn bộ nhớ, tăng thêm có thể làm máy nghẹt hơn. Xem active/idle, queue wait, transaction duration, slow query, locks, buffer cache hit, disk latency và network. Một query chậm có thể do plan đổi khi dữ liệu tăng; có thể do storage shared bị backup job chiếm IOPS; có thể do lock từ migration. Thử một thay đổi có đo trước/sau. Khi thay index, tính write amplification và thời gian tạo index, nhất là bảng lớn.

Câu hỏi: replica có thay backup không? Không. **CPU DB thấp nhưng API timeout?** Chờ lock/I/O/network/pool vẫn có thể làm timeout. **RPO 0 với async replication?** Không bảo đảm trong lỗi primary nếu giao dịch chưa tới replica. **Backup thành công nghĩa restore được?** Chưa; cần restore drill và kiểm tra dữ liệu/app. Nguồn: PostgreSQL backup/restore, MySQL replication.

Một transaction lỗi có thể không làm CPU cao

API gọi **UPDATE notes SET ... WHERE id=?** rồi treo. DB CPU chỉ 20%, nhưng một transaction khác giữ lock trên row/table lâu; request chờ lock, pool app đầy, proxy trả 504. Bằng chứng cần: active sessions, wait event/lock graph, transaction age, query đang chạy và request trace. Nếu chỉ tăng CPU/RAM, lock không biến mất. Cần xác định transaction giữ lock và vì sao: migration, batch job, code quên commit, isolation level hoặc thiếu index khiến lock phạm vi rộng hơn. Giảm tác động có thể hủy một query/transaction theo runbook sau khi xét rollback dữ liệu, rồi sửa app/migration.

Backup/restore theo mốc thời gian

Full backup lúc 00:00, binary log/WAL lưu liên tục, lỗi xóa bảng lúc 14:35. Muốn phục hồi trước 14:35, restore full vào instance cách ly, replay log đến thời điểm/position phù hợp, kiểm tra dữ liệu và tính nhất quán, sau đó quyết định chuyển traffic hoặc trích dữ liệu cần phục hồi. Đừng replay tới sau lệnh DELETE rồi ngạc nhiên dữ liệu vẫn mất. Đồng hồ/timezone và log retention cần rõ. Với MySQL/PostgreSQL, công cụ/cú pháp khác nhau; bài này dạy quy trình và câu hỏi kiểm chứng, không thay runbook version cụ thể.

Lag replication và đọc sau ghi

App ghi primary rồi đọc replica ngay; nếu replica lag 3 giây, request đọc có thể không thấy note mới. Một lựa chọn là đọc primary cho giao dịch vừa ghi, hoặc chờ token/position đồng bộ theo cơ chế engine, hoặc chấp nhận eventual consistency và thông báo UX. Không giải bằng **sleep(1)** cố định. Alert replication lag phải đo theo giây/byte và ảnh hưởng nghiệp vụ; lag 10 giây có thể chấp nhận báo cáo nhưng không chấp nhận xác nhận giao dịch tài chính.

Tự kiểm tra phỏng vấn

Vì sao unique constraint tốt hơn check trùng ở app? Hai request đồng thời có thể cùng thấy chưa tồn tại; constraint ở DB quyết định cuối cùng trong transaction. **Vì sao backup file DB bằng cp khi server đang ghi nguy hiểm?** Có thể copy các trang ở thời điểm khác nhau; dùng backup API/snapshot nhất quán theo engine. **Index nhiều có luôn tốt?** Không; mỗi ghi phải cập nhật index, tăng storage và có thể thay query plan. **Failover có thể gây split-brain?** Nếu hai node cùng nhận write sau partition mà không có cơ chế leader/fencing phù hợp, dữ liệu phân kỳ.

Bạn đạt phần này khi phân biệt được latency do CPU, I/O, lock, connection pool và replication, đồng thời mô tả được một restore có mốc thời gian và phép kiểm ứng dụng. Nguồn: PostgreSQL monitoring, MySQL backup/recovery.

Bài tập giải thích WAL/binlog: một full backup lúc 00:00 cho bạn một điểm bắt đầu, còn log thay đổi cho phép tiến gần thời điểm cần khôi phục nếu log được giữ và tương thích. Nếu log 10:00-11:00 bị mất, bạn không thể giả định replay liên tục qua khoảng ấy. Viết lịch backup/log retention khớp RPO, đặt bản sao ở failure domain khác và diễn tập restore tới mốc ngẫu nhiên. Đây là cách biến "có backup" thành khả năng phục hồi đo được.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://dev.mysql.com/doc/refman/8.4/en/backup-and-recovery.html
- https://www.postgresql.org/docs/current/